

Java ArrayList

Complete Class Notes — Concepts, Pseudocode & Dry Runs

Session notes for tomorrow's class. Covers what ArrayList is, how it differs from a plain array, its internal working, every core method, iteration styles, sorting, and the key interview patterns — each with pseudocode and a worked dry run.

Prepared by Akshit Sharma | 8510968871 | akshubhardwaj231@gmail.com | akshbuilds.tech

Contents

- 1. What is an ArrayList?
- 2. ArrayList vs Array — Key Differences
- 3. Declaring, Creating & Generics
- 4. Internal Working (Dynamic Resizing)
- 5. Core Methods (add, get, set, remove, contains, size)
- 6. Iterating an ArrayList (for, for-each, Iterator, forEach)
- 7. Searching in an ArrayList
- 8. Sorting an ArrayList
- 9. Removing Elements Safely While Iterating
- 10. ArrayList of Custom Objects
- 11. Converting Between Array and ArrayList
- 12. 2D ArrayList (List of Lists)
- 13. Time Complexity Cheat-Sheet
- 14. Common Mistakes & Interview Tips

1. What is an ArrayList?

ArrayList is a resizable array implementation of the `List` interface, part of `java.util`. Unlike a plain array, it can **grow or shrink** automatically as elements are added or removed, and it can only hold **objects** (not primitives directly — primitives get auto-boxed, e.g. `int` becomes `Integer`).

```
import java.util.ArrayList;
import java.util.List;

List<Integer> numbers = new ArrayList<>();
numbers.add(10);
numbers.add(20);
numbers.add(30);
System.out.println(numbers); // [10, 20, 30]
```

2. ArrayList vs Array — Key Differences

Feature	Array	ArrayList
Size	Fixed at creation	Grows/shrinks dynamically
Data type	Primitives & objects	Objects only (auto-boxing)
Syntax	<code>arr.length</code> , <code>arr[i]</code>	<code>list.size()</code> , <code>list.get(i)</code>
Performance	Slightly faster (no overhead)	Small overhead from resizing/boxing
Built-in methods	Very few (via <code>Arrays</code> class)	Many (<code>add</code> , <code>remove</code> , <code>sort</code> , <code>contains...</code>)
Multi-dimensional	Natural (<code>int[][]</code>)	Needs <code>List<List<T>></code>

3. Declaring, Creating & Generics

```
// Declare with the interface type (best practice)
List<String> names = new ArrayList<>();

// With an initial capacity hint (still grows if needed)
List<String> names2 = new ArrayList<>(50);

// From an existing collection
List<String> copy = new ArrayList<>(names);

// Diamond operator <> lets the compiler infer the generic type
```

Why generics?

`List<Integer>` guarantees at compile time that only `Integer` objects go in — no runtime `ClassCastException` later, and no manual casting needed when reading elements out.

4. Internal Working (Dynamic Resizing)

Internally, `ArrayList` is backed by a plain **`Object[]` array**. When it's full and a new element is added, Java allocates a **new, bigger array** (commonly 1.5x the old capacity), copies all existing elements into it, and the old array becomes garbage. This is why repeated single insertions have an **amortized $O(1)$** cost even though occasional resizes are $O(n)$.

Dry Run — capacity growth while adding 5 elements to a list starting at capacity 2

Add #	size before	capacity before	Resize triggered?	capacity after
1	0	2	no	2
2	1	2	no	2
3	2	2	yes -> grow to 3	3
4	3	3	yes -> grow to 4 (approx)	4
5	4	4	yes -> grow to 6 (approx)	6

Exact growth factor is an implementation detail (~1.5x), but the amortized analysis and the $O(1)$ average-case `add()` are what matter for interviews.

5. Core Methods

Method	Purpose	Example
<code>add(x)</code>	append to end	<code>list.add(10)</code>
<code>add(i, x)</code>	insert at index i	<code>list.add(1, 99)</code>
<code>get(i)</code>	read element at index i	<code>list.get(0)</code>
<code>set(i, x)</code>	overwrite element at index i	<code>list.set(0, 5)</code>
<code>remove(i)</code>	remove by index (int arg)	<code>list.remove(0)</code>
<code>remove(Object)</code>	remove first matching value	<code>list.remove(Integer.valueOf(5))</code>
<code>size()</code>	number of elements	<code>list.size()</code>
<code>contains(x)</code>	true/false membership check	<code>list.contains(5)</code>
<code>indexOf(x)</code>	first index of x, or -1	<code>list.indexOf(5)</code>
<code>isEmpty()</code>	true if size == 0	<code>list.isEmpty()</code>
<code>clear()</code>	remove all elements	<code>list.clear()</code>
<code>sort(cmp)</code>	sort in place	<code>list.sort(null)</code>

Pitfall: `remove(int)` vs `remove(Object)`

```
List<Integer> list = new ArrayList<>(List.of(10, 20, 30));
list.remove(1); // removes INDEX 1 -> [10, 30]
list.remove(Integer.valueOf(10)); // removes VALUE 10 -> [30]
```

6. Iterating an ArrayList

```
// 1) classic for loop (index-based)
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i));
}

// 2) for-each (cleanest for read-only traversal)
for (int v : list) {
    System.out.println(v);
}

// 3) Iterator (needed if you must remove() while looping)
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
    int v = it.next();
    if (v % 2 == 0) it.remove(); // SAFE removal
}

// 4) forEach with a lambda
list.forEach(v -> System.out.println(v));
```

7. Searching in an ArrayList

```
FUNCTION linearSearch(list, target):
FOR i FROM 0 TO list.size() - 1:
IF list.get(i) == target: RETURN i
RETURN -1
```

```
// Or simply: list.indexOf(target) / list.contains(target)
```

For a **sorted** ArrayList, `Collections.binarySearch(list, target)` gives $O(\log n)$ lookup, same idea as binary search on arrays.

8. Sorting an ArrayList

```
List<Integer> nums = new ArrayList<>(List.of(5, 3, 8, 1, 9));

Collections.sort(nums); // ascending, natural order
nums.sort(Comparator.reverseOrder()); // descending

// Custom objects: sort by a field using a Comparator
List<String> names = new ArrayList<>(List.of("Charlie", "Alice", "Bob"));
names.sort(Comparator.comparing(String::length)); // by length
names.sort((a, b) -> a.compareTo(b)); // alphabetically
```

Dry Run — Collections.sort([5,3,8,1,9])

Pass idea	Result
Java uses TimSort internally (stable, $O(n \log n)$)	[1, 3, 5, 8, 9]

Output: [1, 3, 5, 8, 9]

9. Removing Elements Safely While Iterating

Modifying a list (add/remove) with a for-each loop or a raw for loop while iterating causes a `ConcurrentModificationException` or skipped elements. Always use an **Iterator**'s own `remove()`, or iterate backwards with an index-based loop, or collect-then-remove.

```
// WRONG - throws ConcurrentModificationException
for (int v : list) {
    if (v == 3) list.remove(Integer.valueOf(3));
}

// RIGHT - iterate backwards
for (int i = list.size() - 1; i >= 0; i--) {
    if (list.get(i) == 3) list.remove(i);
}

// RIGHT - use removeIf (cleanest, Java 8+)
list.removeIf(v -> v == 3);
```

10. ArrayList of Custom Objects

```
class Student {
    String name;
    int marks;
    Student(String name, int marks) { this.name = name; this.marks = marks; }
    public String toString() { return name + ":" + marks; }
}

List<Student> students = new ArrayList<>();
students.add(new Student("Riya", 88));
students.add(new Student("Aman", 72));

students.sort(Comparator.comparingInt(s -> s.marks)); // ascending by marks
System.out.println(students); // [Aman:72, Riya:88]
```

11. Converting Between Array and ArrayList

```
// Array -> ArrayList
Integer[] arr = {1, 2, 3};
List<Integer> list = new ArrayList<>(Arrays.asList(arr));

// ArrayList -> Array
List<Integer> list2 = new ArrayList<>(List.of(1, 2, 3));
Integer[] arr2 = list2.toArray(new Integer[0]);
```

Pitfall

`Arrays.asList(arr)` returns a **fixed-size** list backed by the array — calling `add()` or `remove()` on it directly throws `UnsupportedOperationException`. Wrap it in `new ArrayList<>(...)` as shown above to get a resizable copy.

12. 2D ArrayList (List of Lists)

```
List<List<Integer>> grid = new ArrayList<>();
for (int i = 0; i < 3; i++) {
    grid.add(new ArrayList<>());
    for (int j = 0; j < 3; j++) {
        grid.get(i).add(i * 3 + j);
    }
}
System.out.println(grid); // [[0,1,2],[3,4,5],[6,7,8]]
```

13. Time Complexity Cheat-Sheet

Operation	Time
<code>get(i)</code> / <code>set(i, x)</code>	$O(1)$
<code>add(x)</code> at end	Amortized $O(1)$
<code>add(i, x)</code> at index / <code>remove(i)</code>	$O(n)$ — shifts elements
<code>contains(x)</code> / <code>indexOf(x)</code>	$O(n)$
<code>Collections.binarySearch</code> (sorted list)	$O(\log n)$
<code>Collections.sort</code> / <code>list.sort</code>	$O(n \log n)$

14. Common Mistakes & Interview Tips

- `IndexOutOfBoundsException` — index must be ≥ 0 and $< \text{size}()$, not $\leq \text{size}()$.
- Confusing `remove(int index)` with `remove(Object o)` for Integer lists.
- Modifying a list while iterating with `for-each` -> use `Iterator.remove()` or `removeSelf()` instead.
- Using `Arrays.asList()` and then trying to add/remove -> wrap in `new ArrayList<>()`.
- Forgetting ArrayList can't hold primitives directly — int autoboxes to Integer, which has a small memory/perf cost.
- Choosing ArrayList when you mostly insert/delete at the front -> a LinkedList (or ArrayDeque) may fit better there.
- Always state Big-O for the specific ArrayList operation you used when explaining a solution in an interview.

End of Notes — see the companion 'ArrayList 20 Practice Questions' PDF for hands-on problems.